Take-home: Full-stack engineer (Next.js / tRPC / Postgres)

We'd rather see a small surface done properly than a half-finished product.

Send it back by the end of Saturday 5 September. What we need is a public GitHub repo (or a zip) with a `NOTES.md` in it, plus a live URL we can open in a browser. Host it wherever you like, a default subdomain is fine. No custom domain needed.

1. Background

We run a marketplace where brands post paid clipping campaigns and creators submit short-form clips (TikTok, Instagram, YouTube). Creators get paid per 1,000 views, up to the campaign budget. There's money in the loop, so correctness matters more than feature count.

You'll build a cut-down version of that flow.

2. Stack

This mirrors what we run in production, so it isn't optional:

- Next.js 15 (App Router), React, TypeScript in strict mode
- tRPC v11 for everything between client and server. No REST route handlers for app data.
- Drizzle ORM on Postgres. Local Docker or Supabase local, whichever you prefer. Ship the compose file or the `supabase/` config.
- TailwindCSS and shadcn/ui
- react-hook-form with Zod, schemas shared by client and server
- Vitest

The rest is up to you. If you pull in something substantial, say why in `NOTES.md`.

3. Data model

At minimum:

- `campaign`: title, platform(s), `payout_per_1k_views` (integer cents), `total_budget` (cents), `status` (`draft`, `active`, `paused`, `completed`), `starts_at`, `ends_at`
- `submission`: campaign, creator, post URL, platform, `status` (`pending`, `approved`, `rejected`, `paid`), `rejection_reason`, timestamps
- `submission_metric`: submission, `captured_at` (date), `views`, `likes`, `comments`. One row per submission per day.
- `user`: id, email, `role` (`admin` or `creator`). No real auth, see 4.1.

Migrations have to be generated with `drizzle-kit` and committed. We read them.

4. What it should do

4.1 Auth

Keep this cheap. A signed cookie holding a `userId` plus a dev-only user switcher is enough. Don't wire up an auth provider.

Server side is a different matter: every procedure enforces role and ownership. A creator shouldn't be able to reach another creator's submissions, including by hand-crafting the input.

4.2 Admin

- Campaign list with pagination, search on title, filter by status. Paginate on the server.
- Create and edit a campaign. RHF + Zod, schema shared with the server.
- Campaign detail with a review queue: pending submissions, approve or reject. Rejecting requires a reason.
- Per campaign, an overview with total approved views, budget spent, budget left, and a chart of daily views across the campaign period. The period will contain days with no metrics.

4.3 Creator

- Browse `active` campaigns.
- Submit a clip URL to a campaign. The URL has to look like a real post URL on one of the campaign's platforms. The same URL can't end up on the same campaign twice.
- A "my submissions" list with status, current views and estimated earnings.

4.4 Budget and payout

- Earnings on an approved submission are `floor(views / 1000) * payout_per_1k_views`, taken from the most recent metric row.
- A campaign never pays out more than `total_budget`. If approving a submission would push it over, the approval fails with a typed error the UI can act on.
- Approvals are first come, first served. If two admins approve at the same moment against a budget that only covers one of them, only one can go through. Write up how you handled it in `NOTES.md`.
- Once the remaining budget reaches zero, the campaign becomes `completed` on its own.

4.5 Metrics ingestion

In production these numbers come from third-party APIs. Here, write a script behind `pnpm ingest` that fakes a daily sync:

- one `submission_metric` row per approved submission per day
- views only ever go up
- running it twice for the same day leaves the data as it was
- if one submission blows up mid-run, the rest still finish and the failure gets reported

5. Tests

We're not after coverage numbers. Write the tests you'd actually want on the parts that can break, and be ready to explain why those. At a minimum we expect the payout math, the budget ceiling, concurrent approvals, access control and a repeated ingest run to be covered.

`pnpm test` has to pass on a clean checkout after the setup steps you documented.

6. `NOTES.md`

Short and concrete:

- setup steps that work on a machine that isn't yours
- how you dealt with concurrent approvals, and what you tried or ruled out along the way
- what you left out on purpose
- the first thing you'd fix given another day
- where you used AI tooling and what you had to correct. We use it daily as well, so this isn't a trick question. We're interested in your judgment over the output.

7. How we read it

Most of our attention goes to the payout and budget logic, the schema and migrations, and the shape of your API and types. The UI we look at for states, accessibility and restraint.

Things that won't earn you anything here: custom design work, real auth, extra features.

8. After that

If we take it further, there's a 45-minute call. We go through the repo with you and then extend it together with a small change, so have it running locally.

Questions are welcome, send them over. If you'd rather not wait for an answer, make a call and note the assumption in `NOTES.md`.